Sol Lee
Wenpeng Yin
CMPSC 448
11 October 2026

LLM Fingerprint: Identifying LLMs from their Responses

1. Problem Definition and Dataset

Today, large language models such as ChatGPT, Claude, and Gemini often generate similar-looking text, but each model may have its own writing patterns. The goal of the project is to test whether a machine learning classifier can identify which LLM generated a response.

For this project, I treat the task as a three-class text classification problem with the following labels: GPT, Claude, and Gemini.

The project focuses on two required research questions:
RQ1: Can we identify which LLM generated a response using only LLM output?
RQ2: Does the user's prompt help identify the LLM? For this question, I compare input only, output only, and input + output.

RQ3 tests whether LLM fingerprints generalize to unseen task categories, and RQ4 looks at characteristics such as response length and formatting.

1.1 Dataset Construction

The final dataset contains 360 responses generated from 120 prompts. Each prompt was sent to GPT, Claude, and Gemini, giving 120 responses from each model.

LLM	Model
GPT	gpt-5.4-mini-2026-03-17
Claude	claude-haiku-5-5
Gemini	gemini-3.6-flash

All responses were collected on October 9, 2026, through the providers' API using single-turn requests, no system prompt, and a maximum output length of 4096 tokens. I originally planned to use Gemini 3.8 Flash, but both 3.8 and 3.7 Flash kept returning high-demand errors, so I used Gemini 3.6 Flash.

The prompts were collected from three public datasets:

Category	Source	Number of Prompts
General Knowledge	databricks-dolly-15k	30
Writing	databricks-dolly-15k	30
Math	GSM8K	30
Coding	MBPP	30

Prompts were only taken from these datasets, and all 360 responses were newly generated for this project.

For this project, the same prompts were sent to all three LLMs because if different models received different types of prompts, then the classifier could learn the prompt topic instead of the LLM's writing style. Using the same prompts was important to keep the task category balanced across all three labels.

General and writing prompts come from the open_qa and creative_writing categories of Dolly. Math and coding prompts come from the test splits of GSM8K and MBPP. I shuffled each category with random seed 42 and took the first 30 prompts. After reading sample prompts, I removed 6 that did not fit, which were one time-sensitive question, four writing prompts that were actually math puzzles, general questions, or coding questions, and one prompt that asked the model about itself.

1.2 Train / Validation / Test Split

To prevent the test set from containing a prompt that the classifier already saw during training, the dataset was split by prompt, not individual response. All three responses to the same prompt are always in the same split.

Split	Prompts	Responses
Train	80	240
Validation	20	60
Test	20	60

Each category contributes 20 training prompts, 5 validation prompts, and 5 test prompts.

2. CNN and LSTM Implementation

To compare results directly, both models use the same preprocessing and dataset splits. All models were implemented in PyTorch.

2.1 Text Preprocessing

Preprocessing Pipeline: raw text -> tokenization -> vocabulary -> token IDs -> padding/truncation -> embedding

Text is lowercased and tokenized into words, numbers, and punctuation. I keep markdown symbols and line breaks because formatting may be part of an LLM's writing style.

The vocabulary is created using the training set, and tokens appearing fewer than two times are mapped to <unk>.

All sequences are padded or truncated to 512 tokens. The prompt and response are joined using a <sep> token for the input + output experiment.

2.2 CNN

The CNN uses 100-dimensional embeddings, convolution kernel sizes of 3, 4, and 5, 100 filters for each kernel size, ReLU activation, max pooling, dropout of 0.5, and a final fully connected layer with three output classes.
The goal of the CNN is to detect short patterns of words or formatting that may be characteristic of one LLM.

2.3 LSTM

The RNN model is implemented as a one-layer LSTM with 100-dimensional embeddings, hidden size 128, dropout 0.5, and a fully connected layer with three output classes.
Packed sequences are used so that padding tokens do not affect the final hidden state.

2.4 Training Setup

Setting	Value
Loss	Cross-entropy
Optimizer	Adam
Learning rate	0.001
Batch size	32
Maximum epochs	20
Random seed	42
Model selection	Best validation accuracy

The best model is selected using the validation set. The test set is only used for final evaluation.

3. Results

3.1 RQ1: Can We Identify the LLM from Its Response?

Using only the generated response, the CNN achieved 88.3% test accuracy, while the LSTM achieved 48.3%. Since there are three balanced classes, random guessing would give approximately 33.3% accuracy.

Model	Validation Accuracy	Test Accuracy	Precision	Recall	F1
CNN	0.967	0.883	0.897	0.883	0.884
LSTM	0.600	0.483	0.480	0.483	0.476

CNN confusion matrix:

True LLM	GPT	Claude	Gemini
GPT	16	1	3
Claude	0	18	2
Gemini	0	1	19

CNN performed especially well on Claude and Gemini. GPT had the lowest recall, with 16 of 20 GPT responses classified correctly.

CNN accuracy by category:

Category	Accuracy
Coding	1.000
Math	0.933
Writing	0.867
General Knowledge	0.733

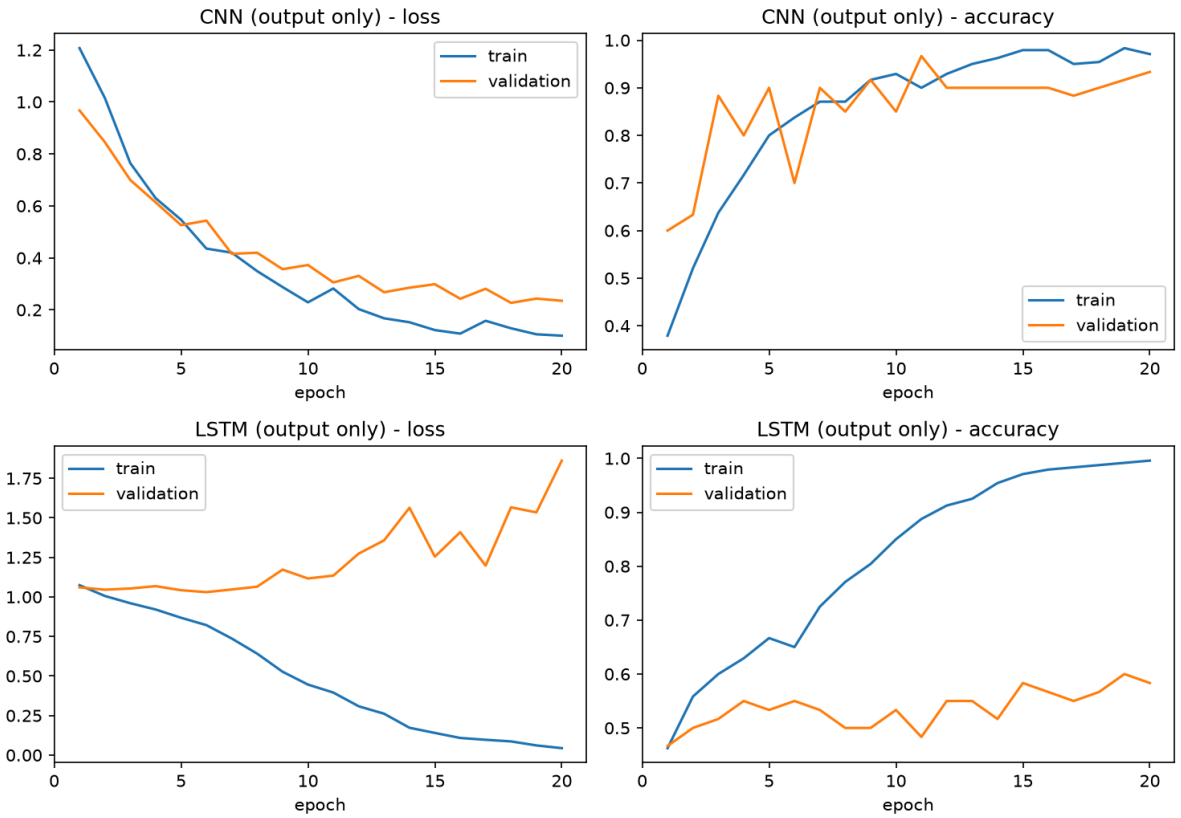
Coding responses were the easiest to identify, while general knowledge responses were the hardest. This may be because short factual answers often look similar in different LLMs.

Because each category contains only 15 test responses, these category-level results should be treated as trends rather than strong conclusions.

3.2 CNN vs. LSTM

The CNN performed much better than the LSTM. The training curves help explain this result. The LSTM reached almost perfect training accuracy, but its validation loss increased after the early epochs, which indicates strong overfitting.

Training and validation loss and accuracy for the output-only CNN and LSTM:



A possible reason may be the size of the dataset because it has only 240 training responses, which is small. The CNN can detect short local patterns with relatively few examples, while the LSTM has to learn longer sequential relationships and seems to memorize the training set instead.

4. RQ2: Does the Prompt Help?

The results for all six required experiments are:

Model	Input	Test Accuracy
CNN	Input Only	0.333
CNN	Output Only	0.883
CNN	Input + Output	0.817
LSTM	Input Only	0.333
LSTM	Output Only	0.483
LSTM	Input + Output	0.467

The input-only classifiers both achieved exactly 33.3% accuracy. Since every prompt appears once with each LLM label, it is expected. For the same prompt, the classifier must make the same prediction each time, so it can only be correct for one of three copies.

This is useful for a sanity check that the classifier is not learning the LLM label from the prompt itself.

Adding the prompt to the response did not improve classification. For CNN, accuracy changed from 0.883 to 0.817, and for LSTM, from 0.483 to 0.467.

Therefore, for this dataset, most of the useful information for identifying the LLM comes from the generated response rather than the user's prompt.

Input-only classification could work well in other datasets if different LLMs received different kinds of prompts. For example, if GPT mostly answered coding questions and Claude mostly answered writing questions, then the classifier could guess the LLM from the topic of the prompt instead of the model's responses. For my dataset, all three LLMs answered the same prompts, so this was avoided.

5. Additional Analysis and Experiments

5.1 RQ3: Does the Fingerprint Generalize Across Tasks?

For this extra-credit experiment, I used a leave-one-category-out setup. The CNN was trained on three categories and tested on the fourth category that it had never seen during training. The test set was all 90 responses from the unseen category, and the validation set only used the other three categories. I used a CNN with output-only, as LSTM already performed poorly in RQ1.

Unseen Category	Test Accuracy
General Knowledge	0.811
Writing	0.767
Math	0.467
Coding	0.811

The classifier still did much better than chance on general, writing, and coding, so some of the LLM-specific patterns seem to work across different task types.
Math was much harder, though. Accuracy dropped to 46.7%, so some of the patterns may be more specific to certain types of tasks.

Looking at each LLM separately, Gemini had the highest and most consistent recall across the unseen categories, ranging from 0.70 to 1.00. Claude was harder to identify on unseen coding and math responses, with recalls of 0.467 and 0.433. GPT performed especially poorly on unseen math responses, with a recall of 0.233.

5.2 RQ4: What Characteristics Distinguish the LLMs?

The three models showed clear differences in response length and formatting. The table shows the average values per response:

	GPT	Claude	Gemini
Words	160.1	208.8	327.4
Lexical diversity	0.77	0.74	0.73

Headers	1.32	0.48	3.05
Bold	4.22	3.58	8.29
Bullet points	4.88	2.80	5.13

Lexical diversity is basically the number of different words compared with the total number of words. I calculated it using the first 100 words of each response. Gemini usually wrote the longest responses and used more headers and bold text, while Claude used less formatting. The lexical diversity scores were very similar for all three models, so it does not seem to help much with separating them.

I removed markdown formatting and line breaks and trained the CNN again to test whether the CNN was mainly detecting formatting.

Data	Test Accuracy
Original responses	0.883
Formatting removed	0.850

Accuracy only dropped by two test examples after removing the formatting. So formatting seems to help a little, but it is not the main reason the classifier works. The model is probably also using things like word choice, phrasing, response length, and sentence structure.

6. Lessons and Experience

The biggest thing I learned from this project was that the way the dataset is built can affect the results just as much as the model itself. Sending the same prompts to every LLM made RQ2 much easier to understand. If each model had received different prompts, the classifier might have learned the topic of the prompt instead of the source LLM.

I also learned that data leakage can happen in small ways. For example, I split the data by prompt and built the vocabulary using only the training set. I also made sure that API-specific metadata, such as the finish reason, was not included in the classifier input.

Another thing I learned was that a more complex sequence model does not always perform better. I expected the LSTM to do fairly well, but it overfit the small dataset while the simpler CNN performed much better.

The small test set also made me more careful about interpreting the results. The difference between 88.3% CNN accuracy and the 33.3% baseline is clear, but smaller differences based on only a few test examples should not be treated as strongly.

7. Conclusion

This project showed that GPT, Claude, and Gemini have some differences in the way they generate responses, and those differences can be learned by a classifier.

The best result came from the CNN, which reached 88.3% accuracy using only the LLM response, compared with a 33.3% random baseline. The LSTM performed much worse and clearly overfit the small dataset.

Using the prompt did not help when all three models received the same set of prompts. The extra-credit experiments also showed that some of the patterns still worked across different task categories, and that markdown formatting alone does not explain the CNN's performance.

Overall, the results show that even a relatively simple text classifier can pick up differences between the responses generated by these three LLMs.